AIML402: Assignment 2 report

Siddharth Barjatya ( )

September 16th, 2025

This document is a submission for Assignment 2 : Task 2 - a report explaining the strategy of my agent and providing results of its evaluation.

Brief Introduction

The game is a multi-agent version of the classic snake game that takes place on a 2D grid where two populations of snakes compete. Each snake has only a limited 7×7 view around its head, which shows where food, friends, and enemies are in the nearby area. Based on this view, the snake chooses one of three possible actions at every turn: turn left, move forward, or turn right. The main goals are to survive as long as possible, eat food to grow longer, and gain an advantage over the other population by attacking enemies or avoiding fatal crashes.

The environment is fixed during evaluation: the grid size is 50×50, with 40 snakes per player, and each game runs for 100 turns. During training, the genetic algorithm can run for up to 500 generations, with each new generation of snakes evolving from the previous one using selection, crossover, mutation, and elitism. Each snake's performance is scored with a fitness function that rewards survival, growth, eating food, and attacking enemies, while penalising crashes, friendly bites, or being bitten. These scores drive the genetic algorithm so that, over time, the population develops smarter behaviours.

The length of each surviving snake is registered for scoring at end of each game. The difference between enemy score and my score is calculated. The game favours survival and length of my snakes over enemy snakes. Killing enemy snakes, especially early in the game, reduces opponent's chance to hurt my snakes and to be added as part of the final score, enabling a higher score for my snakes. The idea is to encode behaviour for maximum impact -

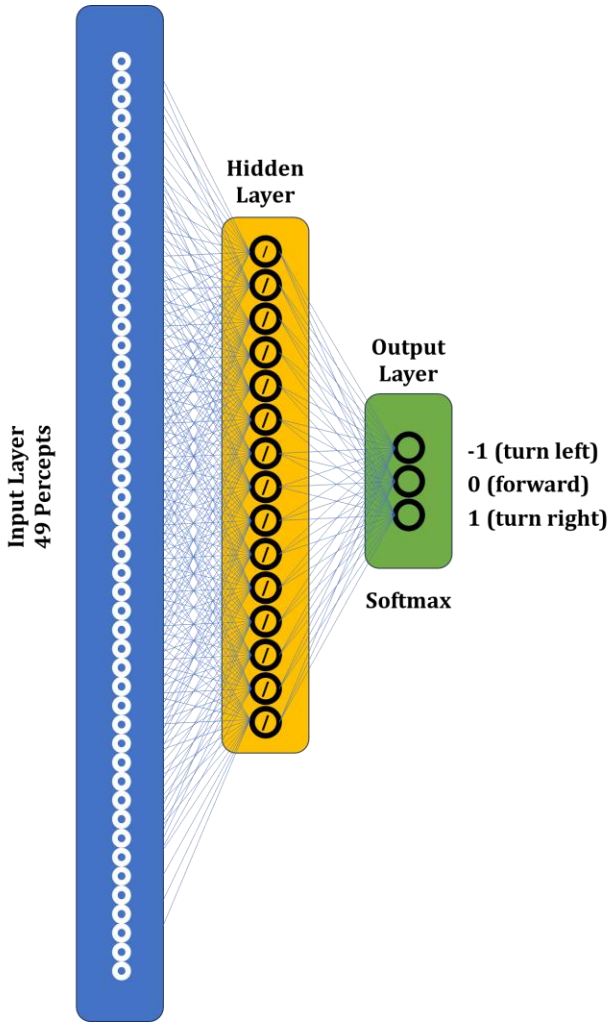
```
# don't die, be alive
# be long, stay long
# be friendly, don't attack friends
# be aggressive, attack enemies
```

How the agent works – the strategy, high level algorithm

My agent works by using a neural network to translate its perception of the environment into actions, and it evolves its strategy over generations through a genetic algorithm. At each turn, the agent observes a 7x7 grid around its head containing information such as food, friends, enemies, and empty spaces. This grid is flattened into 49 percepts and passed into a neural network with one hidden layer of 15 neurons. The network weights and biases are encoded in a chromosome, which acts as the genetic representation of the agent. The outputs of the network are three probabilities corresponding to turning left, moving forward, or turning right, and the action with the highest probability is executed. Each generation consists of a population

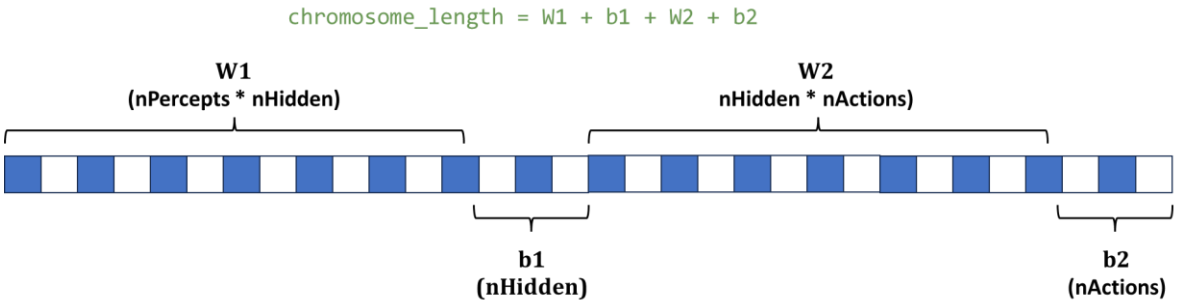
of snakes, and their performance is evaluated through a fitness function that rewards survival, growth, eating food, and attacking enemies, while penalizing friendly attacks, crashes, or being bitten.

NN Architecture & Hidden Layer: The choice of a single hidden layer with 15 neurons was made to balance expressiveness with computational efficiency. Since my agent itself does not learn during a single game as its improvement is guided by the genetic algorithm, the hidden layer acts like a feature layer to encode behaviour for converting percepts into actions.



Elitism, Crossover & Mutation: The evolution process uses elitism to preserve the top six performers, roulette wheel selection to choose parents, single-point crossover to combine genetic material, and mutation to introduce small variations. Hyperparameters such as a 10% mutation probability and a 1-2% gene-level mutation rate were chosen to maintain genetic diversity without destabilizing good solutions. Through these design choices, my agent gradually evolves towards strategies that favour staying alive, growing longer, exploiting opportunities against enemies, and avoiding harmful actions toward friends.

Chromosome Length: The chromosome length is calculated as (nPercepts × nHidden) + nHidden + (nHidden × nActions) + nActions, which in this case becomes (49 × 15) + 15 + (15 × 3) + 3 = 813. This structure encodes W1, b1, W2, and b2 sequentially, allowing efficient crossover and mutation operations during evolution. The chromosome is initialized to have a random but uniform value between (-1, 1) as this helps add/subtract a small change during mutation without skewing the weights.



Softmax Activation: The choice of softmax as the activation function in the output layer was intentional because the agent faces a multi-class decision problem, where only one of three mutually exclusive actions can be taken at a time. Softmax ensures that the outputs are normalized into probabilities that sum to one, which makes the action selection interpretable and stable, while also preventing numerical overflow by subtracting the maximum value during exponentiation. The hidden layer itself uses a linear activation, allowing softmax at the output to shape the final decision.

calculate_survivals() function: Balance of incentives drives the agent toward cautious but aggressive play, rewarding both survival and active engagement with the environment. To support this, the helper function calculate_survivals is used to measure survivals after crashes by checking, turn by turn, whether the snake remained alive despite colliding with another snake. This ensures that not only immediate kills but also resilience after collisions is factored into the agent's evaluation. Each active kill reduces enemy snake's opportunity to strike or score points.

Hyperparameter Tuning: My algorithm uses the following parameters that are optimized to maximize performance - nHidden, elite_count, crossover type, mutation_probability, mutation_rate, choice of fitness function. The method used below is Grid Search, while truncating wherever bas results are observed in order to find the best set of hyperparameters (the external configuration values set before training a model)

Choosing Hidden Layer
(Given crossover = single point, mutation_probability = 0.1, mutation_rate = 0.01, fitness = 2 x meanSize)

Try	nHidden	elite_count	crossover	mutation_probability	mutation_rate	fitness	Win Rate
1	5	0	single point	0.1	0.01	(2*meanSize)	80%
2	10	0	single point	0.1	0.01	(2*meanSize)	20%
3	**15**	0	single point	0.1	0.01	(2*meanSize)	**100%**
4	20	0	single point	0.1	0.01	(2*meanSize)	60%
5	25	0	single point	0.1	0.01	(2*meanSize)	60%
6	**30**	0	single point	0.1	0.01	(2*meanSize)	**100%**

Choosing Elite Count
(Given nHidden = 15)

Try	nHidden	elite_count	Win Rate	Avg Score (Seed 0)	Avg Score (Seed 1)	Avg Score (Seed 2)	Avg Score (Seed 3)
1	15	2	93%	82	46.4	-	-
2	15	4	85%	32	82.8	20.4	-
3	15	**6**	**100%**	**70.6**	**53.6**	**74.8**	**84.4**

4	15	8	80%	-	-	-	-
5	15	10	60%	-	-	-	-

Choosing Mutation Probability
(Given nHidden = 15, elite_count = 6)

Try	mutation_probability	Win Rate (Seed 0)	Win Rate (Seed 1)	Win Rate (Seed 2)	Win Rate (Seed 3)	Win Rate (Seed 4)
1	0.05	80%	80%	80%	100%	60%
2	**0.1**	**80%**	**100%**	**80%**	**100%**	**80%**
3	0.2	20%	0%	-	-	-
4	0.3	60%	0%	-	-	-

Choosing Mutation Rate
(Given nHidden = 15, elite_count = 6, mutation_probability = 0.1)

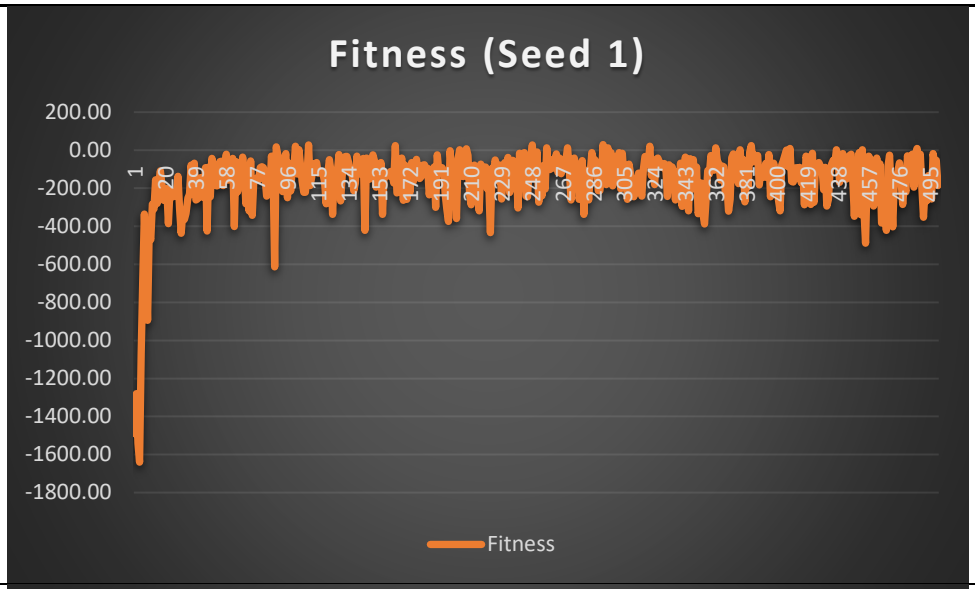
Try	mutation_rate	Win Rate (Seed 0)	Win Rate (Seed 1)	Win Rate (Seed 2)	Win Rate (Seed 3)	Win Rate (Seed 4)
1	**0.01**	**100%**	**100%**	**100%**	**40%**	**80%**
2	**0.02**	**100%**	**100%**	**100%**	**100%**	**100%**
3	0.03	100%	100%	40%	100%	100%

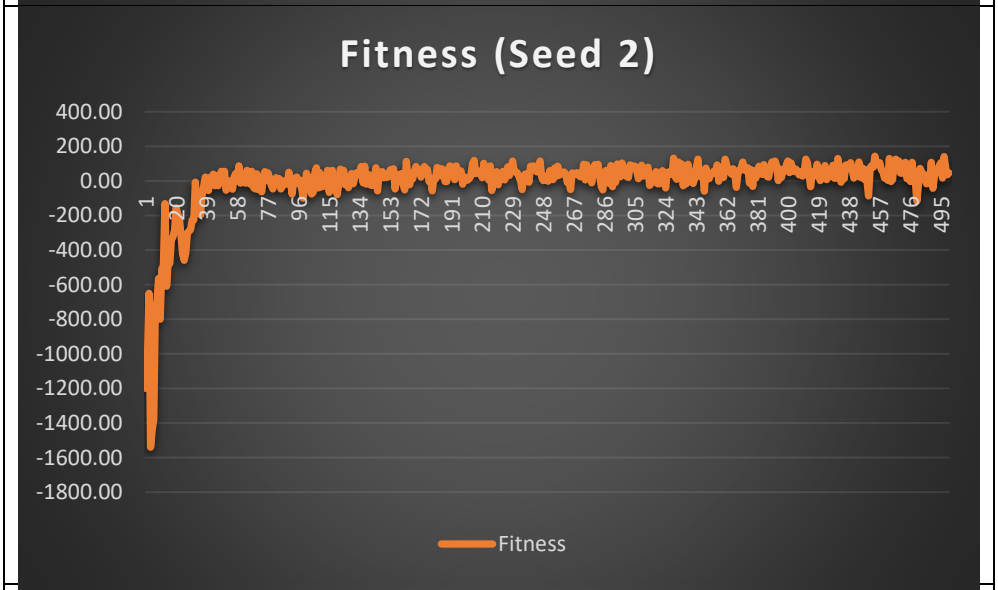
Analysis of the agent's performance

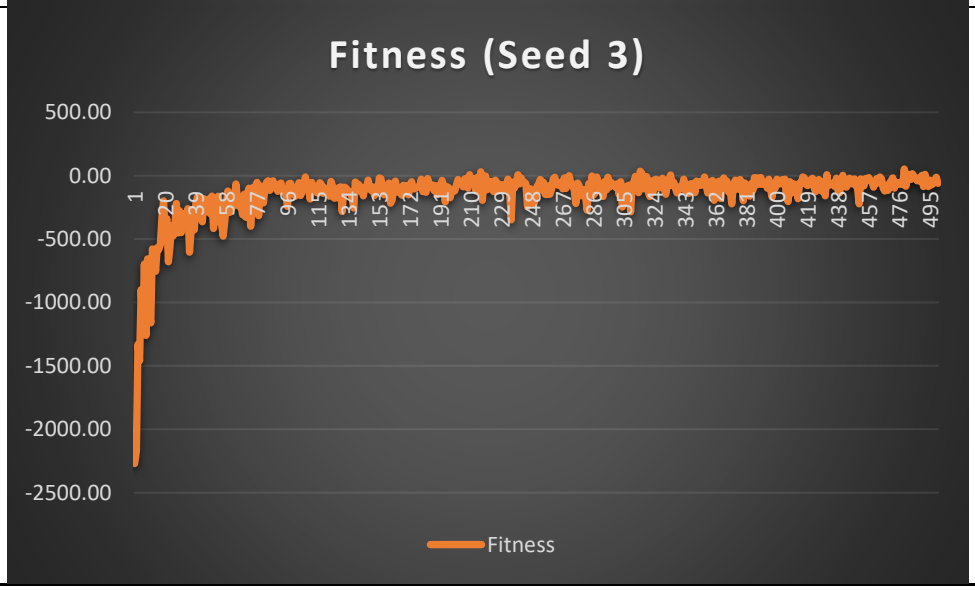
Through an arbitrary hit & trial method, I found encoding the following fitness function to be providing quite good outcomes (although not optimal) in letting the agent snakes learn the expected behaviour –

fitness[n] = 100*isAlive + 2*meanSize + 100*enemiesChomped
- 100*friendsChomped - 100*total_friend_attacks + 10*total_enemy_attacks + 10*total_foods - 10*total_bitten

Fitness graphs for the above function is expressed across multiple seeds by running 500 generation training schedule. Fitness function follows a typical pattern of a genetic algorithm that shows how the population evolves over generations. In the early generations, fitness is generally low and fluctuates widely due to random initialization of chromosomes and high diversity, representing the exploration phase. As the algorithm progresses, good genes begin to combine through crossover and mutation, leading to a rapid rise in fitness. Eventually, the curve starts to plateau, reflecting slower improvements as the population converges and diversity decreases. Occasional jumps may still appear, but over time, the fitness stabilizes, indicating fine-tuning of solutions.

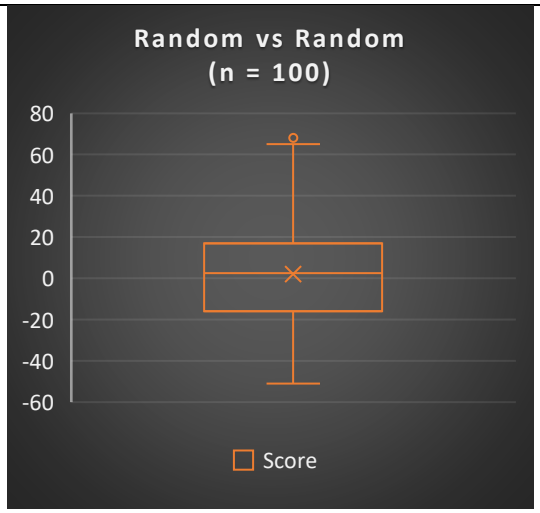

Fitness (Seed 1)
200.00
0.00
-200.00
-400.00
-600.00
-800.00
-1000.00
-1200.00
-1400.00
-1600.00
-1800.00
1
20
39
58
77
96
115
134
153
172
191
210
229
248
267
286
305
324
343
362
381
400
419
438
457
476
495
Fitness


Fitness (Seed 2)
400.00
200.00
0.00
-200.00
-400.00
-600.00
-800.00
-1000.00
-1200.00
-1400.00
-1600.00
-1800.00
1
20
39
58
77
96
115
134
153
172
191
210
229
248
267
286
305
324
343
362
381
400
419
438
457
476
495
Fitness


Fitness (Seed 3)
500.00
0.00
-500.00
-1000.00
-1500.00
-2000.00
-2500.00
1
20
39
58
77
96
115
134
153
172
191
210
229
248
267
286
305
324
343
362
381
400
419
438
457
476
495
Fitness

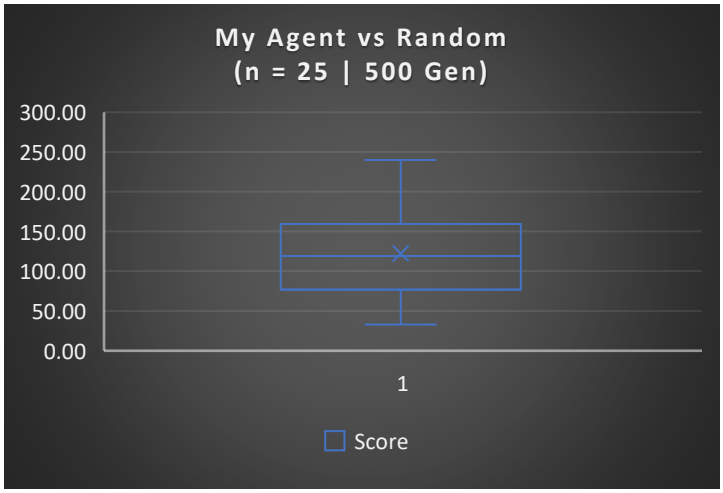


In order to analyse my agent, I first established a baseline score against which I can be confident about the results. In order to do so based on 100 game simulations, I was able to arrive at +/- 20 score range with most scores falling within +/- 60 points with a mean score of 0. Hence in order to beat a random agent a consistent score of 60 or higher is required to assess my agent's performance.



Seed	Score 1	Score 2	Score 3	Score 4	Score 5	Win Rate
1	123	45	110	33	118	**100%**
2	70	240	119	206	209	**100%**
3	145	86	156	102	162	**100%**
4	65	84	98	40	53	**100%**
5	138	182	173	148	157	**100%**

When performance is observed across multiple initial game seeds, compared to the random vs random score range my agent outperforms the score range by a clear margin. My agent shows a performance range of +/- 40 score range with most scores falling within 70 to 160 points with a mean score of 120. Hence my agents beats the random agent by a considerable margin.



Conclusion – summary/discussion of the results

The GA evolves snakes over up to 500 generations, optimizing a fitness function that rewards survival, growth, eating food, and attacking enemies, while penalizing crashes and friendly attacks. Hyperparameters such as hidden layer size, elite count, mutation probability, and mutation rate were tuned using a grid search across multiple seeds. Experiments showed that

a 15-neuron hidden layer, elite count of 6, 10% mutation probability, and 1–2% mutation rate provided optimal performance.

Performance evaluation demonstrated that the agent consistently outperforms random agents. Fitness evolution graphs followed the expected GA pattern: low and fluctuating fitness during early exploration, rapid improvement as good genes combine, and a plateau during convergence with occasional improvements from mutation. Across multiple game seeds, the agent achieved a mean score of 120, significantly above the baseline random range, confirming that it learns effective survival and combat strategies while avoiding harmful actions toward friends.

Conclusion: The agent successfully translates environmental perception into effective actions via a GA-trained NN. Careful selection of hyperparameters, chromosome encoding, and fitness function allowed the snakes to evolve robust strategies, demonstrating that even a simple NN with a hidden layer can achieve complex multi-agent behavior through evolutionary learning.